

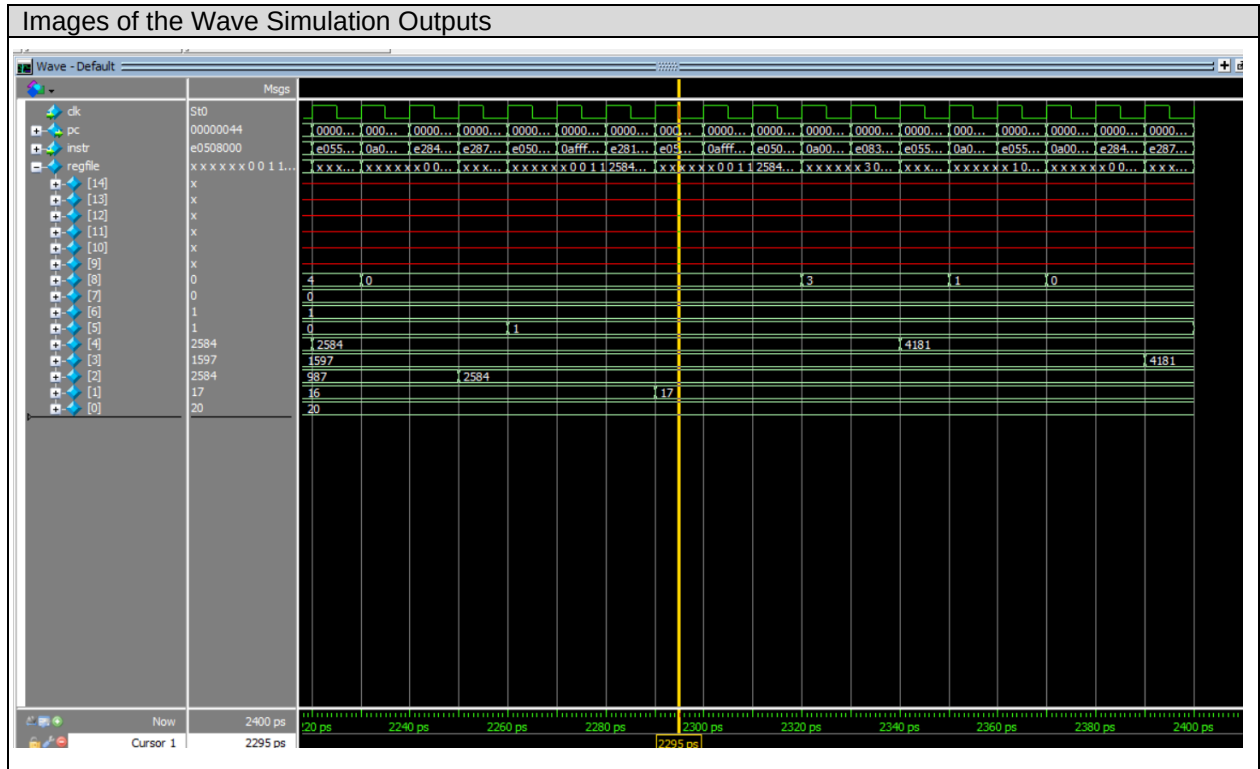


Name:	Lyka Sophia S. Jardeleza	Date:	05/18/2025
Program:	BSCpE	Section:	A361

A. Coding Exercise:

(Please include the following: **1.** Photos and videos of the simulation outputs, **2.** Photos of the codes that were inputted in the ModelSim or any code editor, and **3.** The raw source code that you made for the coding exercise.)

1. Simulation Exercise: Specific Output #1



2. Raw Source Code (System Verilog): Specific Output #2

```
// arm_single.sv
// David_Harris@hmc.edu and Sarah_Harris@hmc.edu 25 June 2013
// Single-cycle implementation of a subset of ARMv4
//
// run 210
// Expect simulator to print "Simulation succeeded"
// when the value 7 is written to address 100 (0x64)

// 16 32-bit registers
// Data-processing instructions
//   ADD, SUB, AND, ORR
//   INSTR<cond><S> rd, rn, #immediate
//   INSTR<cond><S> rd, rn, rm
//   rd <- rn INSTR rm           if (S) Update Status Flags
//   rd <- rn INSTR immediate   if (S) Update Status Flags
//   Instr[31:28] = cond
//   Instr[27:26] = op = 00
//   Instr[25:20] = funct
//   [25]:      1 for immediate, 0 for register
//   [24:21]:   0100 (ADD) / 0010 (SUB) /
//             0000 (AND) / 1100 (ORR)
//   [20]:      S (1 = update CPSR status Flags)
//   Instr[19:16] = rn
//   Instr[15:12] = rd
//   Instr[11:8]  = 0000
```



```
// Instr[7:0] = imm8 (for #immediate type) /
// {0000,rm} (for register type)
//
// Load/Store instructions
// LDR, STR
// INSTR rd, [rn, #offset]
// LDR: rd <- Mem[rn+offset]
// STR: Mem[rn+offset] <- rd
// Instr[31:28] = cond
// Instr[27:26] = op = 01
// Instr[25:20] = funct
// [25]: 0 (A)
// [24:21]: 1100 (P/U/B/W)
// [20]: L (1 for LDR, 0 for STR)
// Instr[19:16] = rn
// Instr[15:12] = rd
// Instr[11:0] = imm12 (zero extended)
//
// Branch instruction (PC <= PC + offset, PC holds 8 bytes past Branch Instr)
// B
// B target
// PC <- PC + 8 + imm24 << 2
// Instr[31:28] = cond
// Instr[27:25] = op = 10
// Instr[25:24] = funct
// [25]: 1 (Branch)
// [24]: 0 (link)
// Instr[23:0] = imm24 (sign extend, shift left 2)
// Note: no Branch delay slot on ARM
//
// Other:
// R15 reads as PC+8
// Conditional Encoding
// cond Meaning Flag
// 0000 Equal Z = 1
// 0001 Not Equal Z = 0
// 0010 Carry Set C = 1
// 0011 Carry Clear C = 0
// 0100 Minus N = 1
// 0101 Plus N = 0
// 0110 Overflow V = 1
// 0111 No Overflow V = 0
// 1000 Unsigned Higher C = 1 & Z = 0
// 1001 Unsigned Lower/Same C = 0 | Z = 1
// 1010 Signed greater/equal N = V
// 1011 Signed less N != V
// 1100 Signed greater N = V & Z = 0
// 1101 Signed less/equal N != V | Z = 1
// 1110 Always any

module testbench();

    logic clk;
    logic reset;

    logic [31:0] WriteData, DataAdr;
    logic MemWrite;

    // instantiate device to be tested
    top dut(clk, reset, WriteData, DataAdr, MemWrite);

    // initialize test
    initial
    begin
        reset <= 1; # 22; reset <= 0;
    end

    // generate clock to sequence tests
    always
```



```
begin
    clk <= 1; # 5; clk <= 0; # 5;
end

// check results
always @(negedge clk)
begin
    if(MemWrite) begin
        if(DataAdr == 100 & WriteData == 7) begin
            $display("Simulation succeeded");
            $stop;
        end else if (DataAdr != 96) begin
            $display("Simulation failed");
            $stop;
        end
    end
end
endmodule

module top(input logic clk, reset,
            output logic [31:0] WriteData, DataAdr,
            output logic MemWrite);

    logic [31:0] PC, Instr, ReadData;

    // instantiate processor and memories
    arm arm(clk, reset, PC, Instr, MemWrite, DataAdr,
            WriteData, ReadData);
    imem imem(PC, Instr);
    dmem dmem(clk, MemWrite, DataAdr, WriteData, ReadData);
endmodule

module dmem(input logic clk, we,
            input logic [31:0] a, wd,
            output logic [31:0] rd);

    logic [31:0] RAM[63:0];

    assign rd = RAM[a[31:2]]; // word aligned

    always_ff @(posedge clk)
        if (we) RAM[a[31:2]] <= wd;
endmodule

module imem(input logic [31:0] a,
            output logic [31:0] rd);

    logic [31:0] RAM[63:0];

    initial
        //$readmemh("memfile.dat",RAM);
        $readmemh("memfile.dat",RAM); // CHANGED THIS

    assign rd = RAM[a[31:2]]; // word aligned
endmodule

module arm(input logic clk, reset,
            output logic [31:0] PC,
            input logic [31:0] Instr,
            output logic MemWrite,
            output logic [31:0] ALUResult, WriteData,
            input logic [31:0] ReadData);

    logic [3:0] ALUFlags;
    logic RegWrite,
          ALUSrc, MemtoReg, PCSrc;
    logic [1:0] RegSrc, ImmSrc;
    logic [2:0] ALUControl; // CHANGED THIS [DONE]
```



```

controller c(clk, reset, Instr[31:12], ALUFlags,
             RegSrc, RegWrite, ImmSrc,
             ALUSrc, ALUControl,
             MemWrite, MemtoReg, PCSrc);
datapath dp(clk, reset,
            RegSrc, RegWrite, ImmSrc,
            ALUSrc, ALUControl,
            MemtoReg, PCSrc,
            ALUFlags, PC, Instr,
            ALUResult, WriteData, ReadData);
endmodule

module controller(input  logic          clk, reset,
                 input  logic [31:12] Instr,
                 input  logic [3:0]  ALUFlags,
                 output logic [1:0]   RegSrc,
                 output logic         RegWrite,
                 output logic [1:0]   ImmSrc,
                 output logic         ALUSrc,
                 output logic [2:0]   ALUControl, // [CHANGED THIS] DONE
                 output logic         MemWrite, MemtoReg,
                 output logic         PCSrc);

    logic [1:0] FlagW;
    logic      PCS, RegW, MemW;

    decoder dec(Instr[27:26], Instr[25:20], Instr[15:12],
               FlagW, PCS, RegW, MemW,
               MemtoReg, ALUSrc, ImmSrc, RegSrc, ALUControl);
    condlogic cl(clk, reset, Instr[31:28], ALUFlags,
               FlagW, PCS, RegW, MemW,
               PCSrc, RegWrite, MemWrite);
endmodule

module decoder(input  logic [1:0] Op,
              input  logic [5:0] Funct,
              input  logic [3:0] Rd,
              output logic [1:0] FlagW,
              output logic      PCS, RegW, MemW,
              output logic      MemtoReg, ALUSrc,
              output logic [1:0] ImmSrc, RegSrc,
              output logic [2:0] ALUControl); // CHANGED THIS DONE

    logic [9:0] controls;
    logic      Branch, ALUOp;

    // Main Decoder

    always_comb
        case(Op)
            // Data processing immediate
            2'b00: if (Funct[5]) controls = 10'b0000101001;
            // Data processing register
            else   controls = 10'b0000001001;
            // LDR
            2'b01: if (Funct[0]) controls = 10'b0001111000;
            // STR
            else   controls = 10'b1001110100;
            // B
            2'b10: controls = 10'b0110100010;
            // Unimplemented
            default: controls = 10'bx;
        endcase

    assign {RegSrc, ImmSrc, ALUSrc, MemtoReg,
           RegW, MemW, Branch, ALUOp} = controls;

    // ALU Decoder
    always_comb

```



```

if (ALUOp) begin // which DP Instr?
    // [CHANGED ADDED EOR AND LSL] DONE
    case(Funct[4:1])
        4'b0100: ALUControl = 3'b000; // ADD
        4'b0010: ALUControl = 3'b001; // SUB
        4'b0000: ALUControl = 3'b010; // AND
        4'b1100: ALUControl = 3'b011; // ORR
        4'b0001: ALUControl = 3'b100; // EOR
        4'b1101: ALUControl = 3'b101; // LSL
        default: ALUControl = 3'bxxx; // unimplemented
    endcase
    // update flags if S bit is set
    // (C & V only updated for arith instructions)
    FlagW[1] = Funct[0]; // FlagW[1] = S-bit
    // FlagW[0] = S-bit & (ADD | SUB)
    FlagW[0] = Funct[0] &
        (ALUControl == 3'b000 | ALUControl == 3'b001);
end else begin
    ALUControl = 3'b000; // add for non-DP instructions
    FlagW = 2'b00; // don't update Flags
end

// PC Logic
assign PCS = ((Rd == 4'b1111) & RegW) | Branch;
endmodule

module condlogic(input logic clk, reset,
                 input logic [3:0] Cond,
                 input logic [3:0] ALUFlags,
                 input logic [1:0] FlagW,
                 input logic PCS, RegW, MemW,
                 output logic PCSrc, RegWrite, MemWrite);

    logic [1:0] FlagWrite;
    logic [3:0] Flags;
    logic CondEx;

    flopenr #(2)flagreg1(clk, reset, FlagWrite[1],
                        ALUFlags[3:2], Flags[3:2]);
    flopenr #(2)flagreg0(clk, reset, FlagWrite[0],
                        ALUFlags[1:0], Flags[1:0]);

    // write controls are conditional
    condcheck cc(Cond, Flags, CondEx);
    assign FlagWrite = FlagW & {2{CondEx}};
    assign RegWrite = RegW & CondEx;
    assign MemWrite = MemW & CondEx;
    assign PCSrc = PCS & CondEx;
endmodule

module condcheck(input logic [3:0] Cond,
                 input logic [3:0] Flags,
                 output logic CondEx);

    logic neg, zero, carry, overflow, ge;

    assign {neg, zero, carry, overflow} = Flags;
    assign ge = (neg == overflow);

    always_comb
        case(Cond)
            4'b0000: CondEx = zero; // EQ
            4'b0001: CondEx = ~zero; // NE
            4'b0010: CondEx = carry; // CS
            4'b0011: CondEx = ~carry; // CC
            4'b0100: CondEx = neg; // MI
            4'b0101: CondEx = ~neg; // PL
            4'b0110: CondEx = overflow; // VS
            4'b0111: CondEx = ~overflow; // VC
        endcase
endmodule

```



```
4'b1000: CondEx = carry & ~zero; // HI
4'b1001: CondEx = ~(carry & ~zero); // LS
4'b1010: CondEx = ge; // GE
4'b1011: CondEx = ~ge; // LT
4'b1100: CondEx = ~zero & ge; // GT
4'b1101: CondEx = ~(~zero & ge); // LE
4'b1110: CondEx = 1'b1; // Always
default: CondEx = 1'bx; // undefined
endcase
endmodule

module datapath(input logic clk, reset,
                input logic [1:0] RegSrc,
                input logic RegWrite,
                input logic [1:0] ImmSrc,
                input logic ALUSrc,
                input logic [2:0] ALUControl, // [CHANGED THIS] DONE
                input logic MemtoReg,
                input logic PCSrc,
                output logic [3:0] ALUFlags,
                output logic [31:0] PC,
                input logic [31:0] Instr,
                output logic [31:0] ALUResult, WriteData,
                input logic [31:0] ReadData);

    logic [31:0] PCNext, PCPlus4, PCPlus8;
    logic [31:0] ExtImm, SrcA, SrcB, Result;
    logic [3:0] RA1, RA2;

    // For handling MOV Rd, Rm, LSL #shamt type instructions CHANGED THIS
    logic [31:0] alu_input_a;
    logic [31:0] alu_input_b;
    logic [31:0] shamt5_extended;
    logic is_mov_immediate_shift;

    // next PC logic
    mux2 # (32) pcmux(PCPlus4, Result, PCSrc, PCNext);
    flopr # (32) pcreg(clk, reset, PCNext, PC);
    adder # (32) pcadd1(PC, 32'b100, PCPlus4);
    adder # (32) pcadd2(PCPlus4, 32'b100, PCPlus8);

    // register file logic
    mux2 # (4) ra1mux(Instr[19:16], 4'b1111, RegSrc[0], RA1);
    mux2 # (4) ra2mux(Instr[3:0], Instr[15:12], RegSrc[1], RA2);
    regfile rf(clk, RegWrite, RA1, RA2,
               Instr[15:12], Result, PCPlus8,
               SrcA, WriteData);
    mux2 # (32) resmux(ALUResult, ReadData, MemtoReg, Result);
    extend ext(Instr[23:0], ImmSrc, ExtImm);

    // ALU logic
    mux2 # (32) srcbmux(WriteData, ExtImm, ALUSrc, SrcB); // CHANGED THIS

    //CHANGED THIS
    assign shamt5_extended = {27'b0, Instr[11:7]};
    assign is_mov_immediate_shift = (ALUControl == 3'b101) && // LSL operation
selected (due to MOV opcode)
based (not immediate for ALU)
(shamt5), not by register Rs
    (ALUSrc == 1'b0) && // Operand2 is register-
    (Instr[4] == 1'b0); // Shift is by immediate

    // Select actual inputs for the ALU
    always_comb begin
        if (is_mov_immediate_shift) begin
            // For MOV Rd, Rm, LSL #shamt:
            // ALU input 'a' should be rf[Rm] (which is WriteData)
            // ALU input 'b' should be shamt5 (from Instr[11:7])
            alu_input_a = WriteData;
```



```
        alu_input_b = shamt5_extended;
    end else begin
        // For standard operations (ADD R0, R1, R2 or ADD R0, R1, #imm) or (LSL R0,
R1, R2)
        // ALU input 'a' is rf[Rn] (which is SrcA)
        // ALU input 'b' is rf[Rm] (WriteData if ALUSrc=0) or ExtImm (if ALUSrc=1)
        alu_input_a = SrcA;
        alu_input_b = SrcB;
    end
end

alu          alu_inst(alu_input_a, alu_input_b, ALUControl,
                    ALUResult, ALUFlags);

endmodule

module regfile(input  logic          clk,
               input  logic          we3,
               input  logic [3:0]    ral, ra2, wa3,
               input  logic [31:0]   wd3, r15,
               output logic [31:0]   rd1, rd2);

    logic [31:0] rf[14:0];

    // three ported register file
    // read two ports combinationaly
    // write third port on rising edge of clock
    // register 15 reads PC+8 instead

    always_ff @(posedge clk)
        if (we3) rf[wa3] <= wd3;

    assign rd1 = (ral == 4'b1111) ? r15 : rf[ral];
    assign rd2 = (ra2 == 4'b1111) ? r15 : rf[ra2];
endmodule

module extend(input  logic [23:0] Instr,
               input  logic [1:0] ImmSrc,
               output logic [31:0] ExtImm);

    always_comb
        case(ImmSrc)
            // 8-bit unsigned immediate
            2'b00: ExtImm = {24'b0, Instr[7:0]};
            // 12-bit unsigned immediate
            2'b01: ExtImm = {20'b0, Instr[11:0]};
            // 24-bit two's complement shifted branch
            2'b10: ExtImm = {{6{Instr[23]}}, Instr[23:0], 2'b00};
            default: ExtImm = 32'bx; // undefined
        endcase
endmodule

module adder #(parameter WIDTH=8)
    (input  logic [WIDTH-1:0] a, b,
     output logic [WIDTH-1:0] y);

    assign y = a + b;
endmodule

module flopenr #(parameter WIDTH = 8)
    (input  logic          clk, reset, en,
     input  logic [WIDTH-1:0] d,
     output logic [WIDTH-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset)    q <= 0;
        else if (en)  q <= d;
endmodule
```



```
module flopr #(parameter WIDTH = 8)
    (input logic clk, reset,
     input logic [WIDTH-1:0] d,
     output logic [WIDTH-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else q <= d;
endmodule

module mux2 #(parameter WIDTH = 8)
    (input logic [WIDTH-1:0] d0, d1,
     input logic s,
     output logic [WIDTH-1:0] y);

    assign y = s ? d1 : d0;
endmodule

module alu(input logic [31:0] a, b,
           input logic [2:0] ALUControl, // [CHANGED THIS] DONE
           output logic [31:0] Result,
           output logic [3:0] ALUFlags);

    logic neg, zero, carry, overflow;
    logic [31:0] condinvb;
    logic [32:0] sum;

    assign condinvb = ALUControl[0] ? ~b : b;
    assign sum = a + condinvb + ALUControl[0];

    // [CHANGED THIS] DONE
    always_comb
        case (ALUControl)
            3'b000: Result = sum; // ADD
            3'b001: Result = sum; // SUB
            3'b010: Result = a & b; // AND
            3'b011: Result = a | b; // ORR
            3'b100: Result = a ^ b; // EOR (NEW)
            3'b101: Result = a << b[4:0]; // LSL (NEW), only use lower 5 bits of 'b'
        endcase

    assign neg = Result[31];
    assign zero = (Result == 32'b0);
    assign carry = (ALUControl == 3'b000 || ALUControl == 3'b001) ? sum[32] : 1'b0;
    assign overflow = (ALUControl == 3'b000 || ALUControl == 3'b001) &&
        ~(a[31] ^ b[31] ^ ALUControl[0]) &
        (a[31] ^ sum[31]);

    assign ALUFlags = {neg, zero, carry, overflow};
endmodule
```

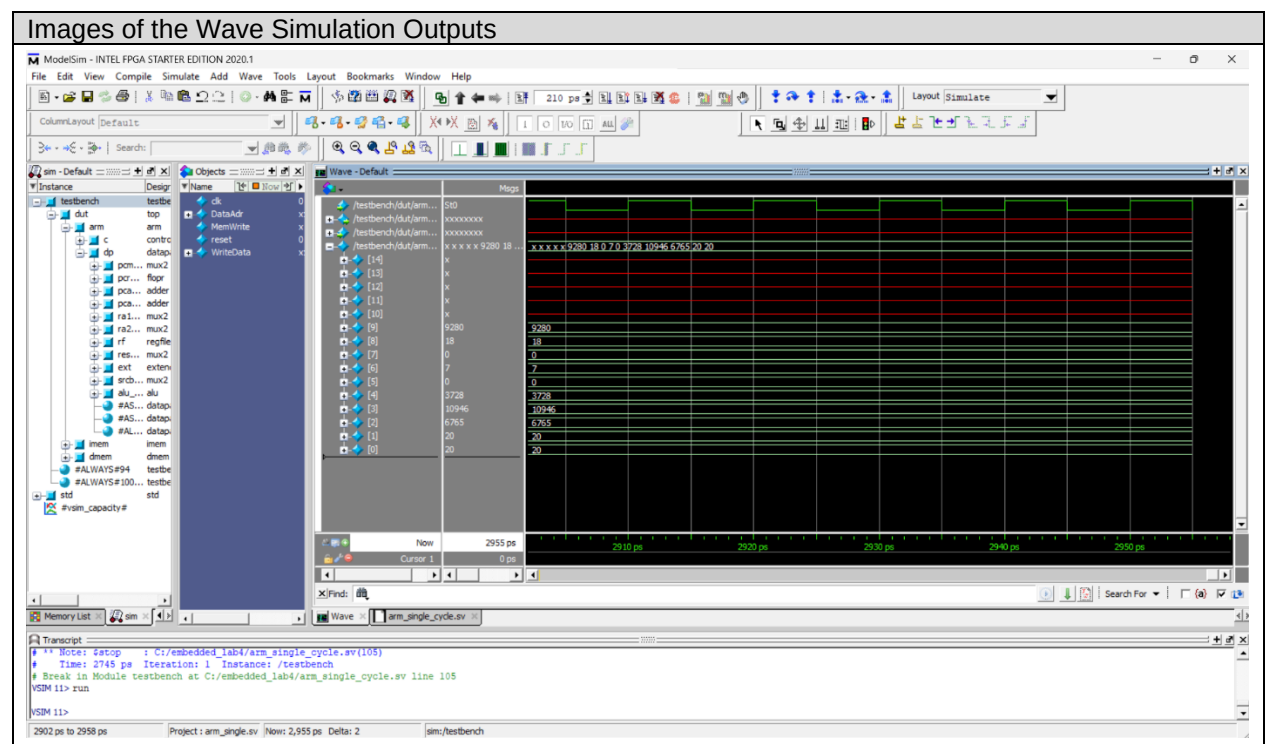
3. Raw Source Code (Machine Code): Specific Output #3

```
E04F700F
E2870014
E2871000
E2875000
E2876001
E2872000
E2823001
E2824000
E2834000
E0508001
0A00000F
E0834002
E0558007
```

```

0A000004
E0558006
0A000006
E2811001
E0508000
0AFFFFF5
E2842000
E2875001
E0508000
0AFFFFF8
E2843000
E2875000
E0508000
0AFFFFF4
E2866006
E5876064
E2878009
E1A08088
E0244008
E1A09488
E2899040
E0244009
  
```

4. Simulation Exercise: Specific Output #4





B. Findings ,Observations, and Comments:

Guiding Questions :

1. *Did you solve the coding and simulation exercises without the help of other/s?*
2. *Are there any difficulties during coding, simulation, and set-up that you encountered?*
3. *What are your findings, observations, and comments in the coding and simulation exercises?*

During the activity, I was able to complete most parts on my own, especially setting up the project and running the initial simulation using the given `memfile.dat`. However, I needed some guidance when editing the SystemVerilog code to add support for the EOR and LSL instructions. One of the challenges I faced was figuring out where to properly insert the new instruction logic in the code and making sure the binary format in `memfile.dat` was correct. I also had to double-check that the waveform showed the right register outputs after the changes. After editing both the Verilog code and `memfile.dat`, the simulation showed that the new instructions worked as expected. The register values like R4, R8, and R9 updated correctly, which confirmed that the EOR and LSL instructions were successfully implemented. Overall, this lab helped me understand more about how instructions work in hardware and how to modify a processor to support new operations.